

CASE 1: Cloud-Native Backend for Food Delivery Company

Case 1:

A food delivery company wants to deploy a cloud-native backend with web APIs, browser clients, and shared cloud storage for receipts and delivery images.

1. Understanding of Case

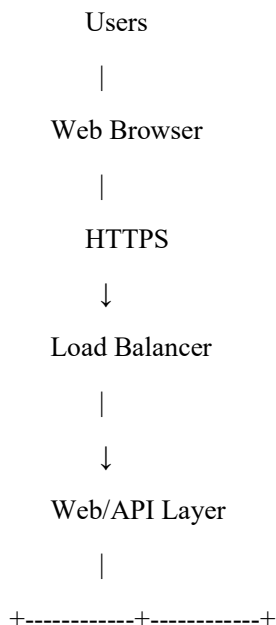
The food delivery platform needs to support customers, restaurants, delivery partners, and administrators. The system should provide:

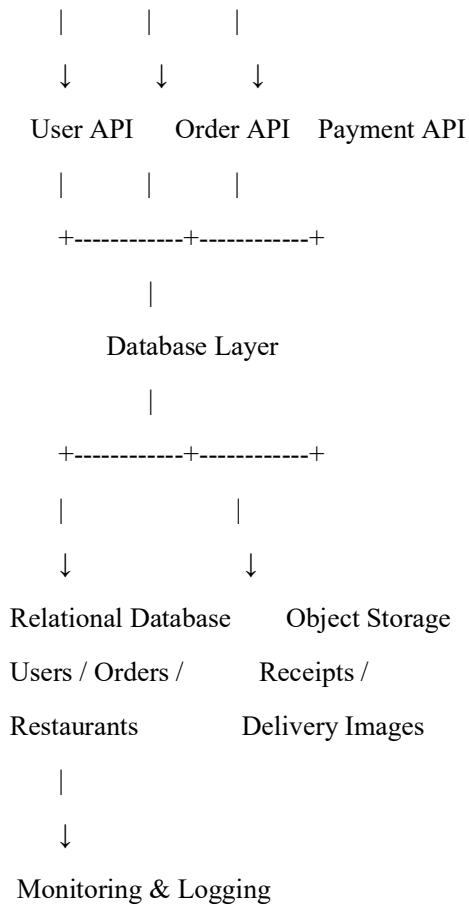
- Browser-based access for users.
- Web APIs for communication between frontend and backend.
- Shared cloud storage for receipts and delivery images.
- Scalable backend services to handle changing numbers of orders.
- Secure access to customer and order information.
- Monitoring and reliable data storage.

A cloud-native architecture is suitable because the workload can change significantly during peak hours.

2. Proposed Architecture

FOOD DELIVERY PLATFORM





3. Role of Each Component

Component	Role
-----------	------

Browser Client	Provides the interface for customers, restaurants, and delivery partners
-----------------------	--

HTTPS	Securely transfers requests between clients and cloud services
--------------	--

Load Balancer	Distributes incoming traffic across backend instances
----------------------	---

Web/API Layer	Handles requests from browser clients
----------------------	---------------------------------------

User API	Manages user registration, login, and profiles
-----------------	--

Order API	Handles food orders, order status, and delivery information
------------------	---

Payment API	Processes payment-related requests
--------------------	------------------------------------

Database	Stores structured data such as users, restaurants, orders, and payments
-----------------	---

Object Storage	Stores receipts, delivery images, and other large files
-----------------------	---

Monitoring	Tracks application health, errors, traffic, and performance
-------------------	---

4. Application of Concepts

The architecture applies several cloud computing concepts:

- **Cloud-native architecture** allows services to scale independently.
- **REST APIs** provide communication between browser clients and backend services.
- **Object storage** is used for large unstructured files.
- **Database services** store structured transactional information.
- **Load balancing** distributes traffic and improves availability.
- **Monitoring** helps identify failures and performance problems.
- **Authentication and authorization** protect customer and order data.

5. Analysis

Using a database for receipts and images would increase database storage requirements. Object storage is more appropriate because receipts and delivery images are unstructured files and may grow rapidly.

A separate API layer also makes the application easier to scale. For example, during lunch or dinner peaks, additional backend instances can be deployed without changing the browser application.

6. Solution Design

The recommended design is:

Browser



HTTPS



Load Balancer



Cloud API Services



Database —————→ Object Storage



Order Data

Images/Receipts



Monitoring & Logging

Conclusion

The proposed cloud-native architecture provides **scalability, security, availability, and efficient storage**. APIs handle application communication, databases store structured information, and object storage provides scalable shared storage for receipts and delivery images.

2: Media Startup – SAN vs Object Storage vs Serverless Processing

Case

A media startup needs to select an appropriate technology for a **video upload workflow** involving SAN-backed storage, object storage, and serverless processing.

1. Understanding of Case

Video files are generally:

- Very large.
- Unstructured.
- Uploaded frequently.
- Accessed by multiple users.
- Suitable for asynchronous processing.
- Potentially required in multiple resolutions and formats.

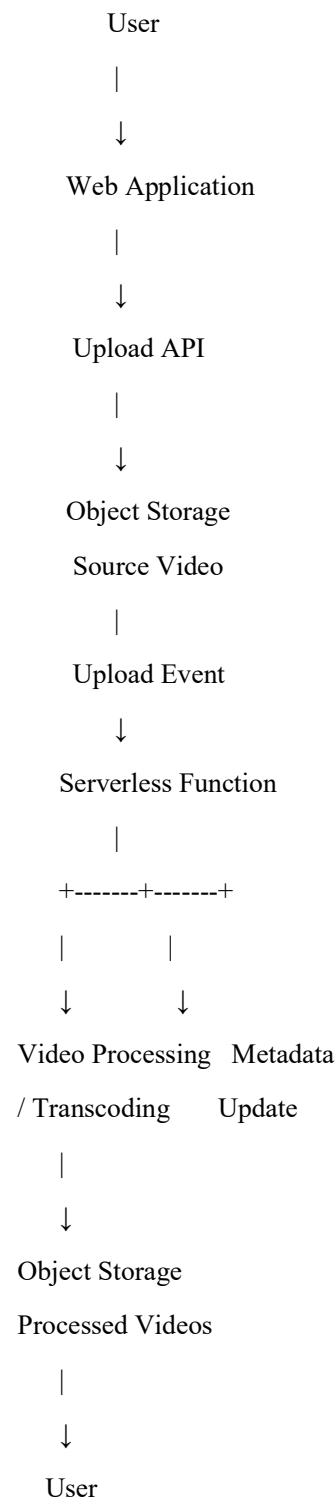
Therefore, the startup needs a solution that provides high scalability, cost efficiency, and automated processing.

2. Comparison

Feature	SAN-backed Storage	Object Storage	Serverless Processing
Main purpose	High-performance block storage	Large unstructured data	Event-based computation
Video storage	Possible	Excellent	Not primarily storage
Scalability	Limited compared with object storage	Very high	Automatic scaling
File sharing	Controlled/block-based	Easy through APIs	Not applicable
Processing	Requires servers	Stores source/output files	Processes uploaded videos
Infrastructure management	Higher	Low	Very low
Cost for large media archive	Higher	Cost-effective	Pay-per-use
Best role	Specialized high-performance workloads	Video storage	Video processing

3. Recommended Architecture

VIDEO UPLOAD WORKFLOW



4. Procedure / Workflow

1. The user opens the media startup's web application.
2. The user selects a video file.
3. The application sends the video to cloud object storage.
4. Object storage generates an upload event.
5. The event triggers a serverless processing function.
6. The function initiates video processing/transcoding.
7. Different versions such as 720p, 1080p, or thumbnails can be generated.
8. Processed files are stored back in object storage.
9. Metadata such as filename, duration, and processing status is stored in a database.
10. The user receives the processed video through the application.

5. Analysis

SAN-backed Storage

SAN provides high-performance block-level storage, but it is not the most suitable choice for a startup storing large volumes of video because infrastructure management and scaling can become expensive.

Object Storage

Object storage is ideal because videos are unstructured objects and storage capacity can scale to very large amounts. It also provides API-based access and is well suited for media distribution workflows.

Serverless Processing

Serverless processing is useful after a video is uploaded. The startup does not need to maintain continuously running processing servers. Processing resources can be invoked when required.

6. Recommended Deployment Model

Recommended model: Object Storage + Serverless Processing

Video Upload



Object Storage



Upload Event



Serverless Processing



Transcoding / Thumbnail



Object Storage



Video Delivery

SAN can be considered only when a specific high-performance block-storage requirement exists.

Conclusion

The best deployment model for the media startup is object storage combined with serverless processing. Object storage provides scalable and economical video storage, while serverless processing automatically handles video-processing workloads based on upload events. This reduces infrastructure management and supports rapid startup growth.

CASE 3: Smart City – Edge and Fog Computing

Case

A smart city platform needs **low-latency decision making near sensors** while still using the cloud for analytics.

1. Understanding of Case

A smart city may contain:

- Traffic cameras.
- Traffic signals.
- Environmental sensors.
- CCTV systems.
- Parking sensors.
- Emergency monitoring devices.

Sending all sensor data directly to the cloud can introduce network latency and consume significant bandwidth.

Therefore, **edge computing and fog computing** can process time-sensitive information closer to where it is generated, while the cloud can perform large-scale analytics.

2. Proposed Architecture

SMART CITY SYSTEM

Sensors / Cameras / IoT Devices

|

↓

EDGE DEVICES

Local Processing

|

Immediate Decisions

|

↓

FOG COMPUTING

Local/Regional Node

|

Aggregation & Filtering

|

↓

CLOUD

Large-Scale Analytics

|

Historical Analysis

|

↓

City Management

3. Edge Computing

Edge computing processes data **directly at or very close to the sensor/device**.

Example:

Traffic Camera

↓

Edge Device

↓

Detect Traffic Congestion

↓

Change Traffic Signal

This is useful when a decision must be made within milliseconds or seconds.

4. Fog Computing

Fog computing provides an intermediate processing layer between edge devices and the cloud.

Example:

Junction 1 —┐

Junction 2 —┤──→ Fog Node —→ Cloud

Junction 3 —┐

The fog node can combine information from several nearby traffic junctions before sending selected data to the cloud.

5. Comparison

Parameter	Edge Computing	Fog Computing	Cloud
Processing location	Near device	Local/regional network	Central data center
Latency	Very low	Low	Higher
Bandwidth usage	Very low	Low	Higher
Local decisions	Excellent	Excellent	Poor for immediate decisions
Multi-device coordination	Limited	Excellent	Excellent
Large-scale analytics	Limited	Limited	Excellent
Example	Traffic signal	City-zone traffic control	City-wide analytics

6. Application of Concepts

For a smart traffic system:

Edge layer

- Detect vehicles.
- Detect congestion.
- Detect accidents.
- Control nearby traffic signals.

Fog layer

- Combine information from multiple intersections.
- Coordinate traffic signals.
- Analyze traffic conditions within a city zone.
- Filter unnecessary sensor data.

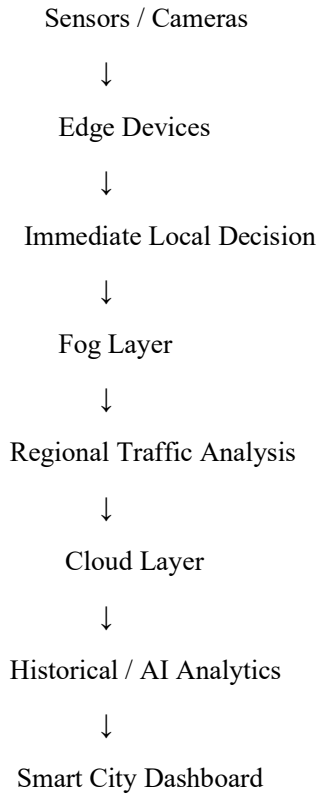
Cloud layer

- Store historical traffic information.
- Perform large-scale analytics.
- Generate traffic predictions.
- Create city-wide reports.
- Train machine-learning models.

7. Solution Design

REAL-TIME LAYER





8. Analysis

If every sensor sends raw data directly to the cloud, network traffic increases and decisions may be delayed due to network communication.

Edge computing solves the latency problem by processing urgent information locally.

Fog computing adds a regional processing layer, making it possible to coordinate multiple edge devices.

The cloud remains important for tasks that do not require immediate decisions, such as historical analysis, reporting, long-term storage, and machine-learning model training.

9. Expected Outcome

Emergency / Real-Time Event



Edge Layer



Immediate Response

Normal Sensor Data



Fog Layer



Data Filtering/Aggregation



Cloud



Long-Term Analytics

Conclusion

A **hybrid Edge + Fog + Cloud architecture** is the most suitable solution for the smart city platform. Edge computing provides extremely low-latency local decisions, fog computing coordinates data from multiple nearby devices, and cloud computing provides powerful large-scale analytics and long-term storage.